

Lab 9 — The Other Bus: SPI, Full-Duplex and Fast

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2 hrs
Introduces	SPI modes, full duplex, DMA-backed transfers
Prerequisites	Lab 5 (bus concepts and register maps).
Beyond the board	One optional jumper for a MOSI-to-MISO loopback.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§9.1 objective**, **§9.2 parts and wiring**, **§9.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

9.1 Objective & TM4C bridge

Drive the third and last of the common serial buses, and understand it as a deliberate set of trade-offs against I²C rather than as an unrelated protocol.

TM4C BRIDGE

Many bare-metal courses stop short of implementing SPI, though the four-wire clocked model with a chip-select line is standard reading. For most people this is the first time they drive it, and the shift-register mental model from a digital-logic course is exactly right: one register rotates bits out while another rotates bits in, on the same clock.

9.2 Parts & wiring

Item	Status	Notes
ESP32-S3 board	On-chip	The lab self-tests with no peripheral attached.
One jumper (optional but recommended)	Required	MOSI to MISO makes this a real end-to-end test.

Item	Status	Notes
microSD module	Optional	A genuine SPI device to talk to, if you want one.

Signal	Pin	Meaning
SCLK	GPIO12	Clock, always driven by the controller
MOSI	GPIO11	Controller Out, Peripheral In
MISO	GPIO13	Peripheral Out, Controller In
CS	GPIO10	Chip select, toggled by the driver

KEY POINT**Make it a real test for free**

As written, the lab clocks bytes out and reads whatever comes back — zeros or floating noise with nothing attached, which proves only that the peripheral runs. Use the Lab 3 trick instead: **jumper MOSI (GPIO11) to MISO (GPIO13)**. Now every byte you send comes straight back, and `rx` should match `tx` exactly. One wire turns a blind transmit into genuine end-to-end verification, and it costs nothing.

NOTE**Analyzer hookup**

Four channels this time: CH0 to SCLK, CH1 to MOSI, CH2 to MISO, CH3 to CS. Sample at 10 MHz and add the SPI decoder set to mode 0. Watching CS fall, the clock burst, and the bytes appear on both data lines simultaneously is the clearest possible demonstration of full duplex.

9.3 How it works

SPI is the third and last of the common serial buses. Assume you know nothing about it; everything you need is below.

The problem every bus solves, and how SPI solves it differently

A microcontroller has to talk to chips that are not inside it — a sensor, a memory, a display. You could give each chip a parallel bundle of eight data wires, and early systems did, but pins are the scarcest resource on a microcontroller and eight wires per device runs out fast. So instead you send the bits **one at a time down a single wire**. That is what "serial" means, and every bus in this series is a different answer to the same follow-up question: *if bits arrive one after another, how does the receiver know where each bit begins?*

Bus	How the receiver knows when to sample	Cost
UART (Lab 3)	It does not get told. Both ends agree on a bit rate in advance and the receiver times the bits itself from the start edge.	Fragile if the rates disagree; limited speed
I²C (Lab 5)	A shared clock wire. The controller pulses SCL and everyone samples SDA on that pulse.	Only two wires, but slow and half duplex

Bus	How the receiver knows when to sample	Cost
SPI (this lab)	A shared clock wire, plus a <i>separate</i> data wire in each direction.	More pins, but fast and full duplex

Because SPI sends a clock, there is no baud rate to agree on and no timing to get wrong — the receiver simply samples whenever the clock says to. That is why SPI can run at tens of megahertz while UART struggles past a few million.

The four wires

Signal	Full name	Driven by	Carries
SCLK	Serial Clock	Controller, always	The timing reference for every bit
MOSI	Controller Out, Peripheral In	Controller	Data going out
MISO	Peripheral Out, Controller In	Peripheral	Data coming back
CS (or SS)	Chip Select / Slave Select	Controller	Which device is being addressed

The names carry the direction, which is genuinely useful: when wiring two boards, **MOSI connects to MOSI and MISO connects to MISO**. You never cross them. Contrast that with a UART, where TX must go to RX and crossing the wires by hand is a classic mistake. SPI's naming removed that whole category of error.

You will still meet the older names **MASTER/SLAVE** in datasheets and in the ESP-IDF function names themselves (`spi_master.h`). The industry has largely moved to controller/peripheral, but the old terms are everywhere in existing documentation, so recognise both.

Chip select: addressing with a wire instead of a protocol

Three wires are SHARED by every device. Only chip-select is per-device — that is how SPI picks who it is talking to, instead of sending an address the way I²C does. N devices needs 3 + N pins.

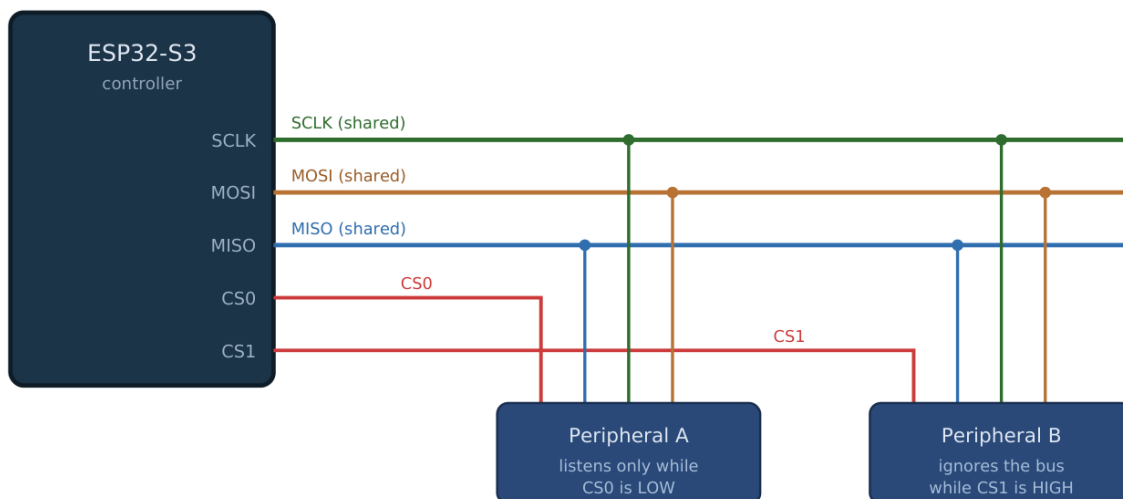


Figure 9.1. SPI topology. SCLK, MOSI and MISO are shared by every device on the bus; only chip select is per-device. A peripheral electrically ignores the bus entirely while its CS is high, which is what makes sharing safe.

SPI bus topology: one controller, two peripherals.

Three wires are SHARED by every device on the bus: SCLK (clock, always driven by the controller), MOSI (controller out, peripheral in) and MISO (peripheral out, controller in). Only chip select is per-device: CS0 runs to Peripheral A, CS1 to Peripheral B.

Peripheral A listens only while CS0 is LOW. Peripheral B ignores the bus entirely while CS1 is HIGH, and tri-states its MISO output so it does not fight the other device for the wire.

That is how SPI picks who it is talking to, instead of sending an address the way I2C does. N devices needs $3 + N$ pins.

This is the structural difference from I²C and it is worth dwelling on. I²C shares two wires and picks a device by **transmitting a 7-bit address** as part of the protocol; every chip listens, compares, and one responds. SPI picks a device by **pulling a physical wire low**. There is no address, no protocol overhead, and no possibility of two devices having the same address.

The trade is pins. Three devices on I²C is still two wires; three devices on SPI is $3 + 3 =$ six. That is the whole comparison in one line: **I²C spends time to save pins, SPI spends pins to save time.**

A peripheral with CS high does not merely ignore the data — it **tri-states** its MISO output, meaning it electrically disconnects, letting go of the wire so another device can drive it. Without that, two peripherals would fight over MISO and you would read garbage.

Full duplex, and why "transmit" also receives

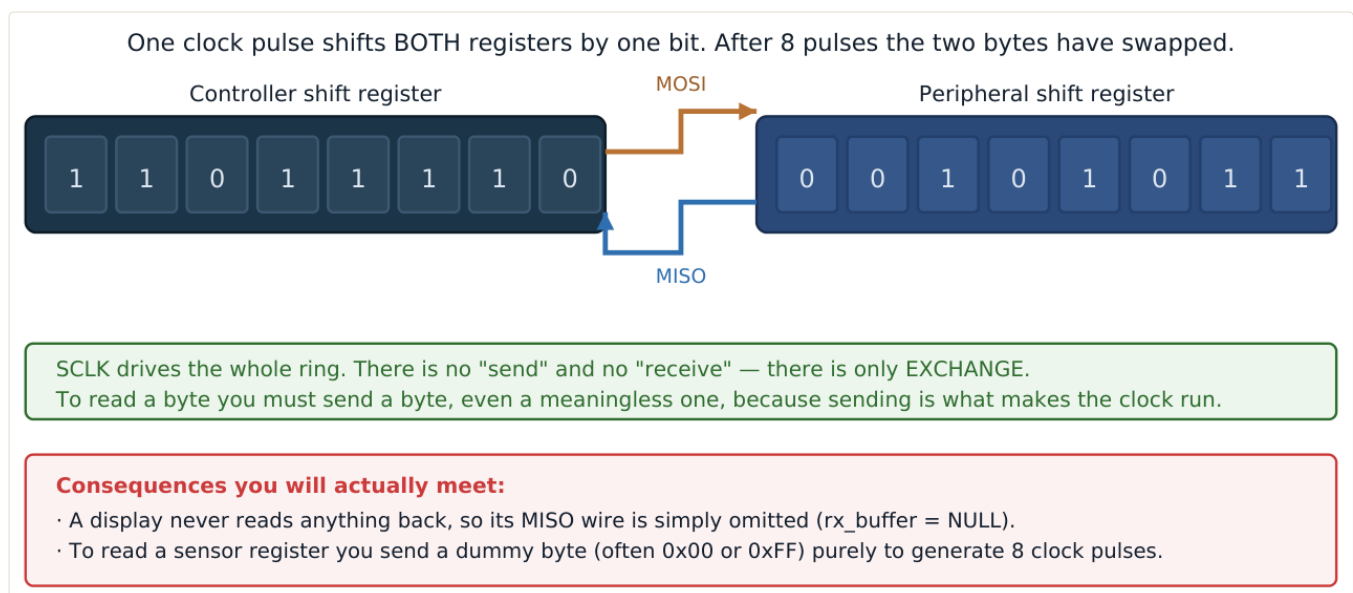


Figure 9.2. The mental model that makes SPI click: two shift registers wired end to end. The clock rotates the whole ring, so after eight pulses the two bytes have swapped places. Sending and receiving are the same operation.

Full duplex shown as two shift registers connected in a ring.

The controller holds one 8-bit shift register and the peripheral holds another. They are connected end to end: the controller's output goes out on MOSI to the peripheral's input, and the peripheral's output returns on MISO to the controller's input.

SCLK rotates the whole ring. One clock pulse shifts BOTH registers by one bit. After eight pulses the two bytes have completely swapped places.

There is no "send" and no "receive" as separate operations; there is only EXCHANGE. To read a byte you must send a byte, even a meaningless one, because sending is what makes the clock run.

Consequences you will actually meet: - A display never reads anything back, so its MISO wire is simply omitted and the driver passes `rx_buffer = NULL`. - To read a sensor register you send a dummy byte, often 0x00 or 0xFF, purely to generate the eight clock pulses that shift the sensor's data out.

UART and I²C are effectively half duplex — data flows one way at a time. SPI is **full duplex**: for every bit clocked out on MOSI, a bit is simultaneously clocked in on MISO. They are not two operations that happen to overlap; they are *one* operation.

The consequence surprises everyone once: **you cannot receive without sending**. The clock only runs while the controller is transmitting, so to read a byte out of a sensor you transmit a dummy byte — often 0x00 or 0xFF — purely to generate the eight clock pulses that shift the sensor's data out. That is why real SPI sensor drivers are full of writes whose transmitted value is meaningless.

And in the other direction: a display never sends anything back, so its MISO pin is often left unconnected entirely and the driver passes `rx_buffer = NULL`. Lab 22 does exactly that.

The four modes, and why getting this wrong is silent

Because SPI sends a clock, both ends must agree on *which edge of that clock carries the data*. There are two independent binary choices, so four combinations, conventionally numbered 0 to 3.

CPOL (clock polarity) — does the clock sit LOW or HIGH when idle? **CPHA** (clock phase) — is data sampled on the *first* edge of each bit period, or the *second*?

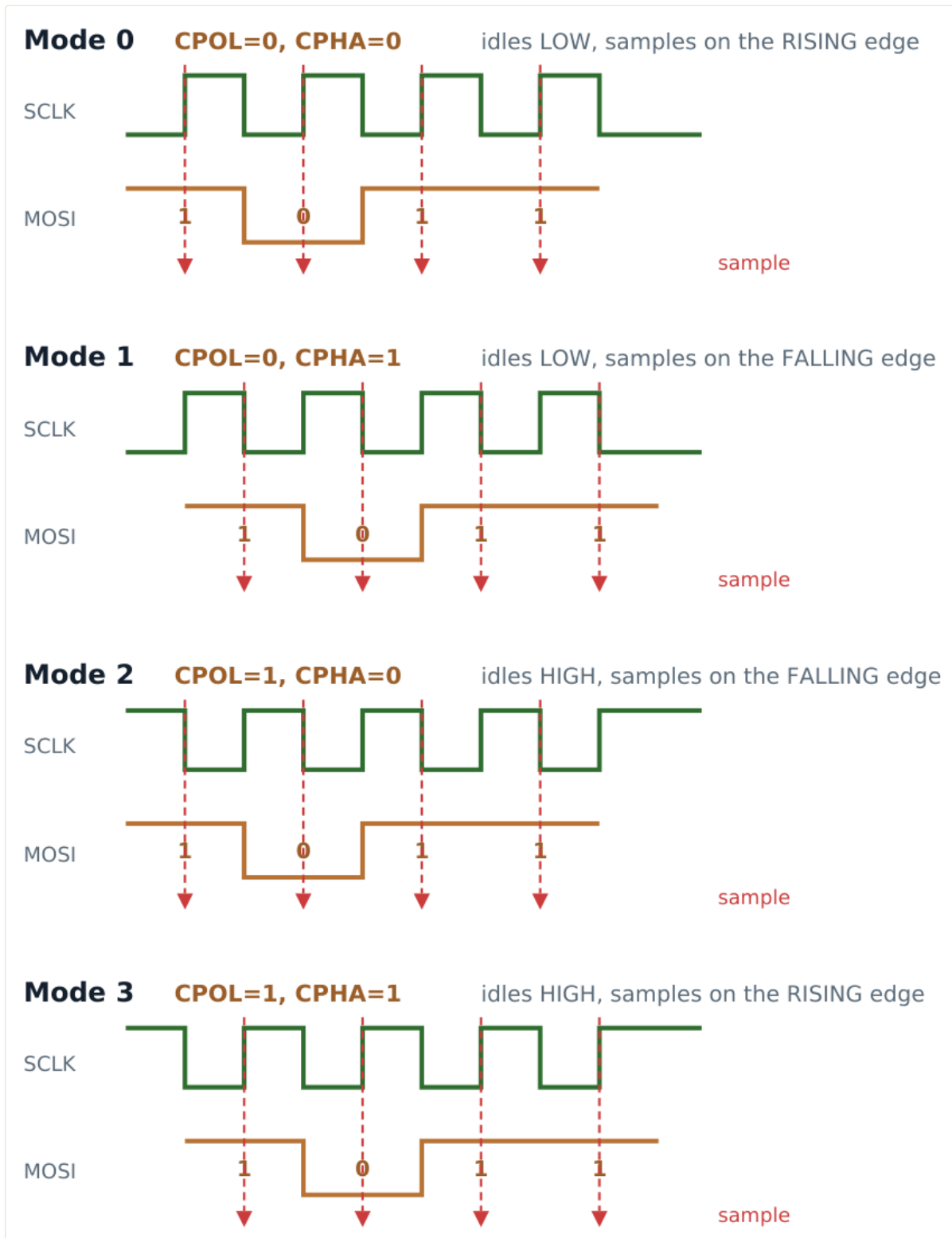


Figure 9.3. The four modes as waveforms. Red dashed lines mark the instant the receiver samples. Notice the pattern: CPOL decides the idle level, CPHA decides whether the sampling edge is the first or second of each cycle. Data must be stable at the sample instant, so the transmitter changes it on the *other* edge.

The four SPI modes as waveforms, with the sampling instant marked.

Two independent binary choices give four combinations. CPOL sets the clock's idle level; CPHA sets whether data is sampled on the first or second edge of each bit period.

Mode	CPOL	CPHA	Clock idles	Data sampled on	Notes
0	0	0	LOW	rising edge	By far the most common. Used by this lab and Lab 22.
1	0	1	LOW	falling edge	Occasional; some ADCs
2	1	0	HIGH	falling edge	Rare
3	1	1	HIGH	rising edge	Second most common; some sensors and displays

In each waveform panel the MOSI data line carries the same bits (1, 0, 1, 1) but is shifted relative to the clock: for CPHA=0 the data changes half a cycle BEFORE the sampling edge so it is already stable; for CPHA=1 it changes ON the first edge and is sampled on the second.

Getting the mode wrong produces NO error. The hardware clocks happily and hands you plausible-looking garbage, often your bytes shifted by one bit position.

Mode	CPOL	CPHA	Clock idles	Data sampled on	Notes
0	0	0	LOW	Rising edge	By far the most common. What this lab and Lab 22 use.
1	0	1	LOW	Falling edge	Occasional; some ADCs
2	1	0	HIGH	Falling edge	Rare
3	1	1	HIGH	Rising edge	Second most common; some sensors and displays

WARNING

The wrong mode gives you no error — just wrong data

This is what makes mode mismatch nasty. If you set the wrong mode, the hardware still clocks happily, the transaction still completes, and `spi_device_transmit` still returns `ESP_OK`. You just sample at the wrong instants and get plausible-looking garbage — often bytes shifted by one bit position, so `0xDE` arrives as `0xBD` or `0x6F`.

How to find the right mode: open the device's datasheet and find its SPI timing diagram. Look at where the clock sits before the first pulse (that is CPOL) and which edge has the data-valid arrow (that is CPHA). Many datasheets simply state "SPI mode 0" in the interface section. This is a thirty-second lookup that prevents an afternoon of confusion, and it is exactly the datasheet-reading skill this series is built on.

How to diagnose it: put the analyzer on it. If the decoder shows bytes that are your data shifted by one bit, you have a CPHA mismatch. If the clock idles at the wrong level, that is CPOL.

What one transaction actually looks like

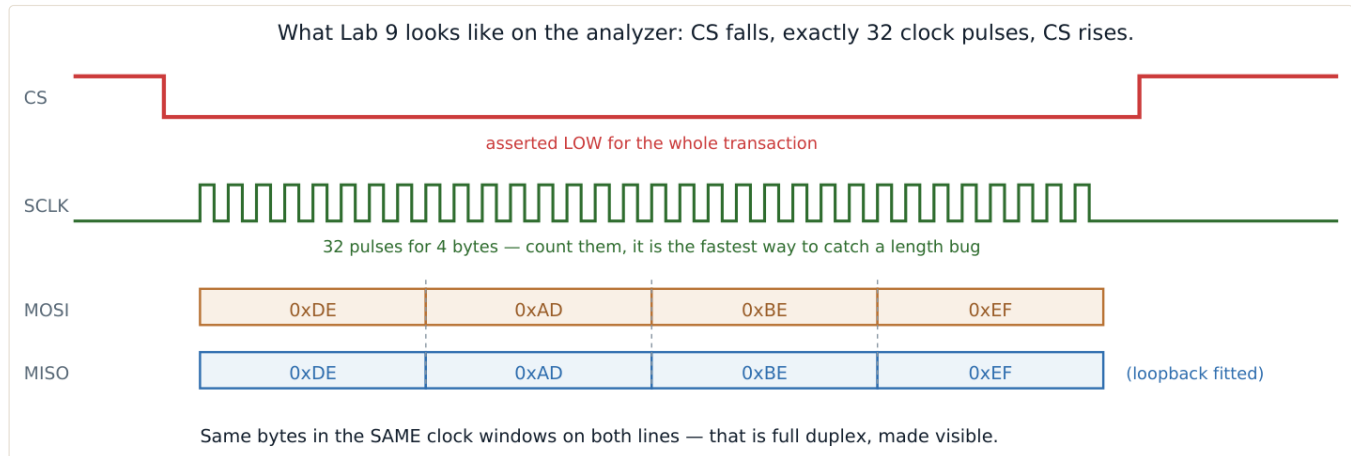


Figure 9.4. The four-byte transaction this lab performs, as it appears on a logic analyzer. Counting clock pulses is the fastest way to catch a wrong transfer length — four bytes must show exactly 32.

One complete four-byte SPI transaction as it appears on a logic analyzer.

Four traces, top to bottom: - CS: idles high, falls before the clock burst, stays LOW for the whole transaction, rises after. - SCLK: exactly 32 pulses, since four bytes is 32 bits. Counting them is the fastest way to catch a wrong transfer length. - MOSI: 0xDE, 0xAD, 0xBE, 0xEF, each byte spanning exactly 8 clock pulses. - MISO: 0xDE, 0xAD, 0xBE, 0xEF with the loopback jumper fitted.

The same bytes appear in the SAME clock windows on both data lines. That is full duplex, made visible.

The sequence never varies: CS falls, the clock runs for exactly as many pulses as there are bits, CS rises. Everything between the CS edges is one atomic transaction as far as the peripheral is concerned — many chips reset their internal state machine on the rising CS edge, which is why you must not release CS partway through a multi-byte command.

That is also why the driver manages CS for you. Getting it right by hand means asserting before the first clock and releasing after the last, and doing it late or early breaks the framing.

Why SPI2 and not SPI1

The ESP32-S3 has several SPI controllers, and they are not interchangeable. **SPI0 and SPI1 are hard-wired to the onboard flash and PSRAM** — they are the path by which your program is fetched from flash and executed. Reconfiguring them for your own use corrupts the running system in ways that look like random crashes. General-purpose use starts at **SPI2**, sometimes called HSPI in older documentation. This is the same class of caution as never touching GPIO26–37.

How fast can you push it?

This lab runs at a deliberately conservative 1 MHz. The peripheral can go far faster — Lab 22 uses 40 MHz to drive a display — and what actually limits you is rarely the chip:

- **The device.** Its datasheet gives a maximum SCLK frequency. Exceeding it produces corrupt data with no warning.

- **Your wiring.** Long breadboard jumpers have capacitance and act as antennas. At 1 MHz nothing cares; at 40 MHz, jumper leads will visibly degrade the edges and you will see corrupted transfers. Short, direct wiring is a real requirement at speed, not fussiness.
- **Whether DMA is involved.** Above a few megahertz the CPU cannot feed the peripheral fast enough byte by byte, which is why `SPI_DMA_CH_AUTO` appears in the setup even though this lab moves only four bytes.

9.4 Procedure & code

```
#include <string.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/spi_master.h"
#include "esp_log.h"

#define PIN_MOSI 11
#define PIN_MISO 13
#define PIN_SCLK 12
#define PIN_CS 10

static spi_device_handle_t dev;

void app_main(void) {
    spi_bus_config_t buscfg = {
        .mosi_io_num    = PIN_MOSI,
        .miso_io_num    = PIN_MISO,
        .sclk_io_num    = PIN_SCLK,
        .quadwp_io_num  = -1,          // -1 = unused (only for quad/octal modes)
        .quadhd_io_num  = -1,
        .max_transfer_sz = 32,
    };
    ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &buscfg, SPI_DMA_CH_AUTO)); // SPI1 is flash

    spi_device_interface_config_t devcfg = {
        .clock_speed_hz = 1 * 1000 * 1000, // 1 MHz
        .mode            = 0,               // CPOL=0, CPHA=0
        .spics_io_num    = PIN_CS,         // driver toggles CS around each transaction
        .queue_size      = 1,
    };
    ESP_ERROR_CHECK(spi_bus_add_device(SPI2_HOST, &devcfg, &dev));

    uint8_t tx[4] = { 0xDE, 0xAD, 0xBE, 0xEF };
    uint8_t rx[4] = { 0 };
    spi_transaction_t t = {
        .length    = 8 * sizeof(tx),      // LENGTH IS IN BITS, not bytes -- classic trap
        .tx_buffer = tx,
        .rx_buffer = rx,
    };
    while (1) {
        memset(rx, 0, sizeof(rx));
        ESP_ERROR_CHECK(spi_device_transmit(dev, &t));
        ESP_LOGI("lab9", "rx: %02X %02X %02X %02X", rx[0], rx[1], rx[2], rx[3]);
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

9.5 Code walkthrough, line by line

Forty-six lines, most of it configuration. Read it as a deliberate contrast with Lab 5's I²C setup: same goal, opposite trade-offs.

Line 1 · `#include`

Standard C, here for `memset` on line 41.

Line 4 · `#include "driver/spi_master.h"`

The SPI controller API. "Master" because the ESP32 drives the clock and decides when transactions happen; the attached chip is the peripheral and only responds. There is a separate `spi_slave.h` for the other role, which you would use if the ESP32 were the one being talked to.

Lines 7-10 · the four pin defines

The four SPI signals, and their names describe the direction unambiguously once you decode them. **MOSI** is Controller Out / Peripheral In, **MISO** is Peripheral Out / Controller In, **SCLK** is the clock the controller drives, and **CS** is chip select. That naming is worth internalising because it means you can wire two devices without thinking: MOSI goes to MOSI, MISO to MISO — the names already encode the crossover, unlike TX/RX on a UART where you must cross the wires yourself.

KEY POINT

All four are on the expansion base

GPIO10 through 13 are in the base's left column, so unlike Lab 8's GPIO0 you can wire this lab entirely through the base. If you are doing the loopback in §9.7, the jumper goes between the 11 and 13 holes.

Line 12 · `static spi_device_handle_t dev;`

The device handle. As with I²C, note there will be **two** levels here: a *bus* (the three shared wires) and a *device* on it (identified by its own chip-select line). This variable is the device.

Lines 15-17 · `.mosi_io_num / .miso_io_num / .sclk_io_num`

The three **shared** signals. Every device on the bus connects to all three; only the chip-select lines are per-device. This is the structural difference from I²C, which shares two wires and distinguishes devices by an address sent *in the protocol*. SPI distinguishes them with a physical wire, which is why it needs more pins but no addressing scheme at all.

Lines 18-19 · `.quadwp_io_num = -1 / .quadhd_io_num = -1`

Two extra data pins used only by quad and octal SPI, where four or eight bits move per clock instead of one. You are doing plain single-bit SPI, so both are unused — and **-1 is the driver's convention for "no pin."** That convention appears throughout ESP-IDF; leaving these at the default of 0 would try to claim GPIO0, which is exactly the kind of quiet mistake that produces baffling behaviour.

Worth knowing this is the same technology your onboard flash uses: the N16R8's flash is QSPI and its PSRAM is Octal SPI, which is why those pins are reserved.

Line 20 · `.max_transfer_sz = 32`

The largest single transfer, in bytes. It matters because the driver sizes its DMA descriptors from it; ask for a transfer larger than this and the call fails. Thirty-two is plenty here and comically small for Lab 22, where one display stripe is 19 KB — that is the same field, sized for a different job.

Line 22 · `spi_bus_initialize(SPI2_HOST, &buscfg, SPI_DMA_CH_AUTO);`

Brings the bus up. Two arguments deserve attention.

SPI2_HOST, and not SPI1. The chip has several SPI controllers and **SPI0 and SPI1 are hard-wired to the onboard flash and PSRAM** — they are how your program gets fetched and executed.

Configuring them for your own use corrupts the running system. General-purpose use starts at SPI2. This is the same "do not touch what the system needs" caution as avoiding GPIO26-37.

SPI_DMA_CH_AUTO lets the driver allocate a DMA channel so large transfers move without the CPU copying each byte. At 32 bytes you will never notice; it is here because the same call in Lab 22 pushes 19 KB per transfer and would be unusable otherwise.

Line 25 · `.clock_speed_hz = 1 * 1000 * 1000`

1 MHz, written as a product so the magnitude reads at a glance; the compiler folds it at compile time. Compare with I²C's 400 kHz ceiling in Lab 5 — SPI is comfortably faster, and 1 MHz here is conservative. Lab 22 runs the same peripheral at 40 MHz.

Like I²C, the speed is **per device**: a slow chip and a fast chip can share one bus, with the driver adjusting per transaction.

Line 26 · `.mode = 0`

SPI mode 0, meaning **CPOL=0, CPHA=0**: the clock idles low, and data is sampled on the rising edge of each pulse. There are four modes, from the two independent binary choices of clock polarity (does the clock idle low or high) and clock phase (is data sampled on the first or second edge of each bit period).

Both ends must agree or every byte is garbage — and, critically, with *no error reported*, because the hardware happily samples at the wrong instants and hands you plausible-looking wrong data. The device's datasheet states which mode it needs, usually in a timing diagram; reading that one line is a thirty-second lookup that prevents an afternoon of confusion. Mode 0 is by far the most common.

Line 27 · `.spics_io_num = PIN_CS`

Hands the chip-select pin to the driver, which then asserts it low before each transaction and releases it after, automatically. **You never toggle CS yourself**. Setting this to **-1** would mean "I will drive CS manually," which you would only do for a device with unusual framing requirements.

A device ignores the bus entirely unless its CS is asserted, which is what lets several devices share MOSI, MISO and SCLK without interfering.

Line 28 · `.queue_size = 1`

How many transactions may be in flight at once. One is fine for the synchronous style used here. Lab 22 sets it to 10 so the CPU can prepare the next display stripe while the current one is still being sent — that overlap is where display throughput comes from.

Line 30 · `spi_bus_add_device(SPI2_HOST, &devcfg, &dev);`

Registers the device on the bus and fills in the handle. The bus and the device are configured separately for the same reason as I²C: one bus, potentially many devices, each with its own speed, mode and chip-select.

Lines 32-33 · `uint8_t tx[4] = { 0xDE, 0xAD, 0xBE, 0xEF }; uint8_t rx[4] = { 0 };`

Four bytes out and four bytes in. `0xDEADBEEF` is a traditional debugging value, chosen because it is instantly recognisable in a hex dump and is very unlikely to occur by accident — if you see it, your data got through; if you see zeros, it did not.

`= { 0 }` zeroes the whole receive array. In C, initialising a struct or array with fewer values than it holds sets every remaining element to zero, so `{ 0 }` is the idiom for "all zeros."

Line 35 · `.length = 8 * sizeof(tx)`

The trap in this lab. The length is measured **in bits, not bytes**. Four bytes is 32 bits, hence the multiply by 8.

Pass `sizeof(tx)` alone and you transmit 4 *bits* — half a byte — and the result is baffling because the transaction still completes successfully. Almost everyone makes this mistake once. The reason the API works in bits is that SPI does not actually require whole bytes; some devices use 9-bit or 12-bit frames, and a bit-count is the only way to express that.

Lines 36-37 · `.tx_buffer = tx; .rx_buffer = rx;`

Where to read from and where to write to. Either may be `NULL`: a `NULL rx_buffer` means "send only, discard what comes back," which is what a display does. Both are supplied here because the point is to see the exchange.

Line 41 · `memset(rx, 0, sizeof(rx));`

Clears the receive buffer before every transaction. Not strictly necessary, but it means any non-zero byte you see afterwards definitely came from *this* transfer rather than lingering from the last one. Deliberately destroying stale state before a test so you cannot be fooled by it is a good habit generally.

Line 42 · `spi_device_transmit(dev, &t);`

Runs the exchange and blocks until it completes. The name is slightly misleading: it does **not** just transmit. SPI is **full duplex**, so for every bit clocked out on MOSI, a bit is simultaneously clocked in on MISO — sending and receiving are one indivisible operation on the same clock edges, not two steps.

The mental model that makes this click is a **shift register ring**: the controller and the peripheral each hold a register, connected end to end, and the clock rotates the whole ring. After eight pulses, whatever was in the controller is in the peripheral and vice versa. You cannot receive without sending; to read a byte from a device you send a dummy byte purely to generate the clock pulses.

With nothing wired to MISO you will read zeros or floating noise, which only proves the peripheral runs. With the loopback jumper from §9.7 fitted, `rx` comes back as `DE AD BE EF` and you have proven the whole path.

Line 43 · `ESP_LOGI("lab9", "rx: %02X %02X %02X %02X", ...);`

`%02X` prints each byte as exactly two uppercase hex digits, zero-padded, so the output columns line up and `0x07` does not print as a bare `7`. Hex rather than decimal because you are looking at raw bytes, where the bit pattern is what matters.

Line 44 · `vTaskDelay(pdMS_TO_TICKS(500));`

Twice a second, slow enough to read and to catch on the analyzer with a generous capture window.

KEY POINT

Watching this on the analyzer

Four channels: CH0 to SCLK (12), CH1 to MOSI (11), CH2 to MISO (13), CH3 to CS (10), plus ground. Sample at **10 MHz** — the ten-times rule against a 1 MHz clock — add the SPI decoder and set it to mode 0. What to look for: CS falls, exactly 32 clock pulses follow, CS rises. Count them. Seeing your four bytes appear on MOSI and MISO in the *same* clock windows is the clearest possible demonstration of full duplex, and it is much more convincing than reading about it.

9.6 Docs & references

DOCS

Trace every line to its source

`spi_bus_config_t`, `spi_bus_initialize`, `spi_device_interface_config_t`, `spi_bus_add_device`, `spi_transaction_t`, `spi_device_transmit` — Peripherals → **SPI Master**. Read the transaction-struct documentation carefully; `.length`'s unit is stated there and is the thing people get wrong.

Which SPI hosts are free — the same page lists which controllers are reserved for flash and PSRAM on each chip. Confirm SPI2 for the S3 yourself.

SPI modes — any protocol reference, but the useful skill is finding the mode in a *device's* datasheet. Look at the timing diagram and see which edge the data is valid on.

`memset` — `<string.h>`.

9.7 Verify, troubleshoot, your turn

Verify. Without the jumper you will see `00 00 00 00` or noise, which only proves the peripheral runs. With MOSI jumpered to MISO you should see exactly `DE AD BE EF`. On the analyzer, confirm CS falls before the clock burst and rises after, and that the byte on MOSI matches the byte on MISO in the same clock window.

Troubleshoot

Symptom	Cause and fix
Only one byte transferred	<code>.length</code> set in bytes instead of bits.
Chip resets or the program corrupts	You used SPI1. Use <code>SPI2_HOST</code> .
Loopback returns zeros	Jumper not seated, or MOSI/MISO defines swapped.
Garbage from a real device	Wrong SPI mode, or the clock is faster than the device supports.

Symptom	Cause and fix
Transfer fails with an error	<code>max_transfer_sz</code> is smaller than the transaction.

Your turn

1. Fit the loopback and confirm the returned bytes match. Then deliberately set `.length` to `sizeof(tx)` instead of `8 * sizeof(tx)` and observe exactly how the output changes — seeing the failure mode makes the trap memorable.
2. Raise the clock to 10 MHz and capture it. Note how much cleaner the loopback still is than you might expect, and how much the wire length starts to matter.
3. If you have the microSD module, read its identification response. A real device that acknowledges you is a genuinely satisfying end to the bus trilogy.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 9.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09 ←	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.